

Practical 11

Purva Tagde

23070521111

Batch B1

Example:

Index.html

```
JavaScript > index.html > html > head > style > .active
30
39 <!DOCTYPE html>
40 <html lang="en">
41 <head>
42 <meta charset="UTF-8">
43 <title>Timers, Dynamic UI & Modular Design</title>
44 <!-- Basic styling for the page -->
45 <style>
46 body { font-family: Arial; padding: 20px; }
47 /* Each class session block */
48 .session {
49 padding: 10px;
50 margin: 5px 0;
51 border: 1px solid #ccc;
52 border-radius: 5px;
53 }
54 /* Highlight active session */
55 .active {
56 background: #c8f7c5;
57 }
58 </style>
59 </head>
60 <body>
61 <h2>Class Scheduler (with Timers & Modules)</h2>
62 <!-- Container for all sessions -->
63 <div id="sessions">
64 <div class="session" data-time="09:00">HTML Basics - 9:00 AM</div>
65 <div class="session" data-time="10:00">CSS Design - 10:00 AM</div>
66 <div class="session" data-time="11:00">JavaScript Logic - 11:00 AM</div>
67 </div>
68 <!-- Display countdown timer -->
69 <p id="countdown"></p>
70 <!-- Main entry JavaScript file (module-based) -->
71 <script type="module" src="main.js"></script>
72 </body>
73 </html>
```

main.js

```
index.html products.json JS lab2.js JS script.js data.json 10_10_25.html pract7_activity_finaltask.html JS main.js x # Info_P Info_P
JavaScript > JS main.js > ...
1 // Import functions from timer.js and ui.js modules
2 import { startCountdown, highlightActiveSession } from './timer.js';
3 import { handleViewportChange, addDynamicSessions } from './ui.js';
4 // Select the countdown display element from the DOM
5 const countdownDisplay = document.getElementById('countdown');
6 // Start countdown for next class (e.g., 5 minutes = 300 seconds)
7
8 startCountdown(300, countdownDisplay);
9 // Highlight the active session every minute automatically
10 setInterval(highlightActiveSession, 60000); // recheck every 60 seconds
11 highlightActiveSession(); // run once immediately on load
12 // Handle screen resizing for responsive behavior
13 window.addEventListener('resize', handleViewportChange);
14 handleViewportChange(); // run on page load as well
15 // Enable event delegation on session elements
16 addDynamicSessions();
```

timer.js

```
ts.json JS lab2.js JS script.js data.json 10_10_25.html pract7_activity_finaltask.html JS main.js # Info_Page.css JS timer.js Info_Page.css
JavaScript > JS timer.js > startCountdown
1 // timer.js - contains countdown and session highlight logic
2 // Function to start a countdown timer
3 export function startCountdown(duration, display) {
4   let time = duration; // time in seconds
5   // setInterval() runs the code every 1 second
6   const timer = setInterval(() => {
7     const minutes = Math.floor(time / 60); // convert seconds to minutes
8     const seconds = time % 60; // remaining seconds
9     // Update countdown message on the page
10
11     display.textContent = `Next session starts in ${minutes}:${seconds < 10 ? '0' + seconds
12     : seconds}`;
13     time--; // decrease the timer each second
14     // Stop timer when it reaches 0
15     if (time < 0) {
16       clearInterval(timer);
17       display.textContent = "Session Started!";
18     }
19   }, 1000); // executes every 1000 milliseconds (1 second)
20 }
21 // Function to highlight the currently active session
22 export function highlightActiveSession() {
23   const sessions = document.querySelectorAll('.session'); // get all session elements
24   const now = new Date(); // get current date/time
25   const currentHour = now.getHours(); // extract current hour (24-hour format)
26   // Loop through all session divs
27   sessions.forEach(session => {
28     const hour = parseInt(session.dataset.time.split(':')[0]); // extract hour from data-time
29     // If current hour matches session time -> highlight
30     if (hour === currentHour) {
31       session.classList.add('active');
32     } else {
33       session.classList.remove('active');
34     }
35   });
36 }
```

ui.js

```
2.js JS script.js {} data.json 10_10_25.html pract7_activity_finaltask.html JS main.js # Info_Page.css JS timer.js JS ui.js
JavaScript > JS ui.js > addDynamicSessions
1 // ui.js - handles dynamic UI and responsive design behavior
2
3 // Function to change background color based on screen size
4 export function handleViewportChange() {
5 // If screen width < 600px, apply a light blue background
6 if (window.innerWidth < 600) {
7 document.body.style.background = '#f0f8ff';
8 } else {
9 // For larger screens, use white background
10 document.body.style.background = 'white';
11 }
12 }
13
14 // Function to add dynamic behavior (event delegation)
15 export function addDynamicSessions() {
16 const sessionsDiv = document.getElementById('sessions');
17 // Add event listener to parent container (event delegation)
18 sessionsDiv.addEventListener('click', (e) => {
19 // Check if clicked element has class "session"
20 if (e.target.classList.contains('session')) {
21 alert("You selected: " + e.target.textContent);
22 }
23 });
24 }
```

Output:

127.0.0.1:5500/JavaScript/index.html

--->

Class Scheduler (with Timers & Modules)

HTML Basics - 9:00 AM
CSS Design - 10:00 AM
JavaScript Logic - 11:00 AM

Next session starts in 4:59

Camera

Task:

index.html

JavaScript > index.html > html > head > style > #countdown

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4  <meta charset="UTF-8">
5  <title>Timers, Dynamic UI & Modular Design</title>
6  <style>
7  body {
8    font-family: Arial, sans-serif;
9    padding: 20px;
10   transition: all 0.3s ease;
11 }
12 .session {
13   padding: 10px;
14   margin: 5px 0;
15   border: 1px solid #ccc;
16   border-radius: 5px;
17   cursor: pointer;
18 }
19 .active {
20   background-color: #c8f7c5;
21 }
22 #countdown {
23   font-size: 18px;
24   font-weight: bold;
25   margin-top: 10px;
26 }
27 #countdown.red {
28   color: red;
29 }
30 button {
31   margin-top: 10px;
32   padding: 8px 12px;
33   background-color: #007bff;
34   color: white;
35   border: none;
36   border-radius: 4px;
```

```
35   border: none;
36   border-radius: 4px;
37   cursor: pointer;
38 }
39 button:hover {
40   background-color: #0056b3;
41 }
42 @media (max-width: 500px) {
43   body {
44     font-size: 14px;
45   }
46 }
47 </style>
48 </head>
49 <body>
50 <h2>Class Scheduler (with Timers & Modules)</h2>
51 <div id="sessions">
52   <div class="session" data-time="09:00">HTML Basics - 9:00 AM</div>
53   <div class="session" data-time="10:00">CSS Design - 10:00 AM</div>
54   <div class="session" data-time="11:00">JavaScript Logic - 11:00 AM</div>
55 </div>
56 <button id="addBtn">Add Session</button>
57 <p id="countdown"></p>
58 <script type="module" src="main.js"></script>
59 </body>
60 </html>
61
```

[helper.js](#)

```
JavaScript > JS helper.js > ...
1
2 export function formatTime(seconds) {
3   const minutes = Math.floor(seconds / 60);
4   const sec = seconds % 60;
5   return `${minutes}:${sec < 10 ? '0' + sec : sec}`;
6 }
7
8 export function logMessage(msg) {
9   console.log(`[LOG] ${msg}`);
10 }
11
```

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF {} JavaScript Port: 5500

05-11-2025 15:24

timer.js

```
JavaScript > JS timer.js > startCountdown > timer > setInterval() callback
1 import { formatTime, logMessage } from './helper.js';
2
3 export function startCountdown(duration, display) {
4   let time = duration;
5   const timer = setInterval(() => {
6     display.textContent = `Next session starts in ${formatTime(time)}`;
7
8     if (time < 60) {
9       display.classList.add('red');
10    } else {
11      display.classList.remove('red');
12    }
13    time--;
14    if (time < 0) {
15      clearInterval(timer);
16      display.textContent = "Session Started!";
17      logMessage("Countdown finished.");
18    }
19  }, 1000);
20 }
21 export function highlightActiveSession() {
22   const sessions = document.querySelectorAll('.session');
23   const now = new Date();
24   const currentHour = now.getHours();
25
26   sessions.forEach(session => {
27     const hour = parseInt(session.dataset.time.split(':')[0]);
28     if (hour === currentHour) {
29       session.classList.add('active');
30     } else {
31       session.classList.remove('active');
32     }
33   });
34 }
35
```

Ln 7, Col 1 Spaces: 4 UTF-8 CRLF {} JavaScript Port: 5500

15:25

ui.js

```
JavaScript > JS ui.js > setupAddSessionButton
1
2 import { logMessage } from './helper.js';
3
4 export function handleviewportChange() {
5   if (window.innerWidth < 600) {
6     document.body.style.background = '#f0f8ff';
7   } else {
8     document.body.style.background = 'white';
9   }
10 }
11
12 export function addDynamicSessions() {
13   const sessionsDiv = document.getElementById('sessions');
14   sessionsDiv.addEventListener('click', (e) => {
15     if (e.target.classList.contains('session')) {
16       alert("You selected: " + e.target.textContent);
17       logMessage("Session clicked: " + e.target.textContent);
18     }
19   });
20 }
21
22 export function setupAddSessionButton() {
23   const button = document.getElementById('addBtn');
24   const sessionsDiv = document.getElementById('sessions');
25
26   button.addEventListener('click', () => {
27     const newDiv = document.createElement('div');
28     newDiv.className = 'session';
29     newDiv.dataset.time = '12:00';
30     newDiv.textContent = 'New Added Session - 12:00 PM';
31     sessionsDiv.appendChild(newDiv);
32     alert('New session added!');
33     logMessage("New session added dynamically.");
34   });
35 }
36
```

Ln 22, Col 1 Spaces: 4 UTF-8 CRLF {} JavaScript Port: 5500

15:25 05-11-2025

main.js

```
JavaScript > JS main.js > ...
1
2 import { startCountdown, highlightActiveSession } from './timer.js';
3 import { handleviewportChange, addDynamicSessions, setupAddSessionButton } from './ui.js';
4
5 const countdownDisplay = document.getElementById('countdown');
6
7 startCountdown(300, countdownDisplay);
8
9 setInterval(highlightActiveSession, 60000);
10 highlightActiveSession();
11
12 window.addEventListener('resize', handleviewportChange);
13 handleviewportChange();
14
15 addDynamicSessions();
16
17 setupAddSessionButton();
18
```

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF {} JavaScript Port: 5500

15:26 05-11-2025

OUTPUT:

Class Scheduler (with Timers & Modules)

HTML Basics - 9:00 AM

CSS Design - 10:00 AM

JavaScript Logic - 11:00 AM

Compiler Constuction - 15:31

Add Session

Next session starts in 4:39



Class Scheduler (with Timers & Modules)

HTML Basics - 9:00 AM

CSS Design - 10:00 AM

JavaScript Logic - 11:00 AM

Add Session

Session Started!



Class Scheduler (with Timers & Modules)

HTML Basics - 9:00 AM

CSS Design - 10:00 AM

JavaScript Logic - 11:00 AM

Add Session

Next session starts in 0:46

